

Regularization and Optimization in Deep Learning

1. Reminder: Neural Network Models

Outline of the Lecture

1. **Reminder: Neural Networks and Backpropagation**
2. **Generalization, Capacity, and Overfitting**
3. **Regularization Techniques**
4. **Optimization in Deep Learning**
5. **Hyperparameter Tuning**

Reminding Neural Networks and Backprop

1. Definition of a Neural Network Model

A neural network is a **parametric function**:

$$h_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}, \quad x \mapsto h_{\theta}(x),$$

where $\theta \in \mathbb{R}^p$ are the parameters (weights and biases).

Objective of learning:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(h_{\theta}(x_i), y_i),$$

where $\ell(\cdot, \cdot)$ measures the prediction error.

The goal of training is to find

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$$

Interpretation:

- $h_{\theta}(x)$ represents a model mapping inputs x to predictions.
- θ gathers all network parameters (matrices $W^{(\ell)}$, biases $b^{(\ell)}$).

2. Neural Networks are (Generally) Non-Linear Models

Let us recall what a **linear model w.r.t. the parameters** means:

$$h_{\theta}(x) = \theta^{\top} \phi(x)$$

It is **linear in θ** , because each parameter θ_k multiplies a fixed basis function $\phi_k(x)$.

In contrast, a neural network is typically defined as:

$$h_{\theta}(x) = f_{\theta_L}^{(L)} \circ f_{\theta_{L-1}}^{(L-1)} \circ \dots \circ f_{\theta_1}^{(1)}(x)$$

with

$$f_{\theta_{\ell}}^{(\ell)}(x) = \sigma(W^{(\ell)}x + b^{(\ell)})$$

→ Each layer applies a non-linear activation σ (e.g. ReLU, tanh, sigmoid).

Hence, $h_{\theta}(x)$ is a non-linear function of both x and θ .

Example (two layers):

$$h_{\theta}(x) = W^{(2)} \sigma(W^{(1)}x + b^{(1)}) + b^{(2)}$$

⇒ nonlinear in θ because of $\sigma(\cdot)$ inside the composition.

4. Gradient Descent

Goal: minimize the loss function

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(h_{\theta}(x_i), y_i)$$

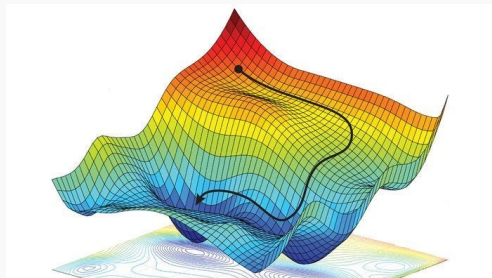
by iteratively updating the parameters θ in the opposite direction of the gradient.

Gradient Descent update rule:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$$

where $\eta > 0$ is the learning rate controlling the step size.

Issue: computing $\nabla_{\theta} \mathcal{L}(\theta)$ on all N samples is computationally expensive for large datasets.



Example of gradient descent trajectory on a loss surface.

5. Gradient Descent — Visual Illustration



Video: Gradient descent trajectory on a simple loss surface.

6. Backpropagation

Objective: compute the gradient of the loss $\mathcal{L}(\theta)$ with respect to all network parameters $\theta = \{W^{(\ell)}, b^{(\ell)}\}_{\ell=1}^L$

Forward pass:

$$\begin{cases} z^{(\ell)} = W^{(\ell)}a^{(\ell-1)} + b^{(\ell)} \\ a^{(\ell)} = \sigma(z^{(\ell)}) \end{cases} \quad \text{for } \ell = 1, \dots, L$$

where $a^{(0)} = x$ and $a^{(L)} = h_{\theta}(x)$.

Chain rule:

$$\frac{\partial \mathcal{L}}{\partial a^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial a^{(\ell+1)}} \frac{\partial a^{(\ell+1)}}{\partial a^{(\ell)}}$$

Each layer's gradient depends on the next layer's gradient.

Backward pass:

$$\begin{cases} \delta^{(L)} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \odot \sigma'(z^{(L)}) \\ \delta^{(\ell)} = (W^{(\ell+1)})^{\top} \delta^{(\ell+1)} \odot \sigma'(z^{(\ell)}) \\ \text{for } \ell = L-1, \dots, 1 \end{cases}$$

Gradients:

$$\nabla_{W^{(\ell)}} \mathcal{L} = \delta^{(\ell)} (a^{(\ell-1)})^{\top}, \quad \nabla_{b^{(\ell)}} \mathcal{L} = \delta^{(\ell)}$$

Parameter update:

$$W^{(\ell)} \leftarrow W^{(\ell)} - \eta \nabla_{W^{(\ell)}} \mathcal{L}, \quad b^{(\ell)} \leftarrow b^{(\ell)} - \eta \nabla_{b^{(\ell)}} \mathcal{L}$$

Overfitting and Underfitting

6. Machine Learning vs Optimization

Optimization aims to minimize the training loss:

$$\mathcal{L}_{\text{train}}(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(h_{\theta}(x_i), y_i)$$

Based on training samples (x_i, y_i) , we aim to minimize the empirical risk:

$$\theta^* \in \arg \min_{\theta} \mathcal{L}_{\text{train}}(h_{\theta}(x_i), y_i)$$

However, this does not guarantee good performance on unseen data.

Generalization is the ability to perform well on new, unseen data. We want models that generalize well beyond the training set, i.e. $h_{\theta}(x_{\text{unseen}})$ to give accurate predictions.

7. The Problem of Overfitting

Model complexity vs generalization:

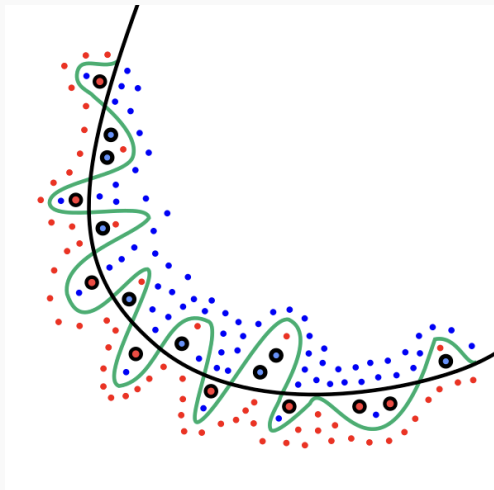
Neural networks can have millions of parameters. They can perfectly fit the training data, even if it contains noise.

Underfitting: model too simple $\rightarrow$ fails to capture patterns. **Overfitting:** model too complex $\rightarrow$ memorizes training data.

Example: fitting N data points with a polynomial of degree 12.

$$\hat{y}(x) = \sum_{k=0}^{12} a_k x^k$$

fits all points but fails on new data.



Overfitting visual example

9. Model Capacity

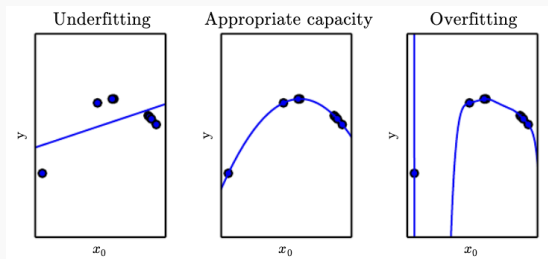
Capacity measures a model's ability to fit a wide variety of functions.

Low capacity: too simple $\rightarrow$ high bias $\rightarrow$ underfitting.

High capacity: too flexible $\rightarrow$ high variance $\rightarrow$ overfitting.

Controlling capacity:

- Choice of **hypothesis space** (e.g., linear vs polynomial functions)
- Number of parameters (for example number of layers and neurons in a NN)



10. Representational vs Effective Capacity

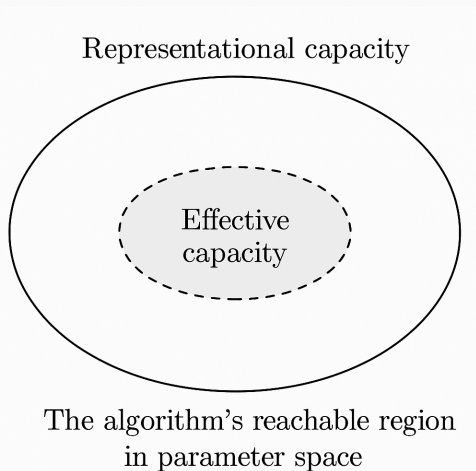
Representational capacity: Set of all functions that the model can represent given its architecture.

$$\mathcal{H}_{\text{model}} = \{f_{\theta} : \theta \in \Theta\}$$

Effective capacity: Subset of this space that can actually be reached by the learning algorithm within practical limits (time, data, optimization).

Reasons for the gap:

- Non-convex optimization $\rightarrow$ local minima.
- Poor initialization or optimizer limitations.
- Implicit regularization of gradient descent.



11. Occam's Razor — The Principle of Parsimony

Philosophical principle:

"Among competing hypotheses that explain the observations equally well, prefer the simplest one."

In machine learning:

- Simple models are less likely to overfit noise.
- Complexity should be increased only if justified by data.

Historical link:

- Origin: William of Ockham (1287–1347)
- Formalized in Statistical Learning Theory (Vapnik & Chervonenkis, 1971)

12. The No Free Lunch Theorem

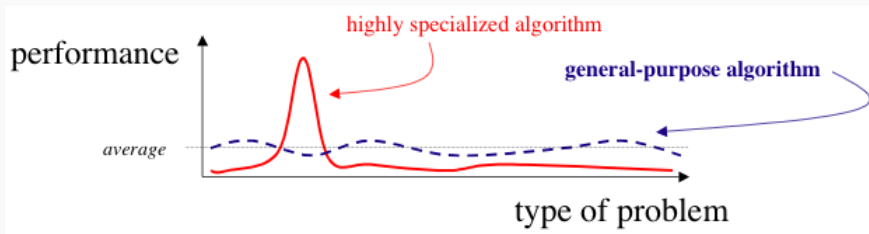
Statement: "any two optimization algorithms are equivalent when their performance is averaged across all possible problems"

Meaning:

- Any algorithm's success depends on the assumptions it makes about the data.

Implication:

- No universal regularizer or architecture.
- We must choose methods suited to the specific task and data structure.



8. Train, Validation, and Test Sets

Goal: estimate how well the model generalizes to unseen data.

We split the dataset into three disjoint subsets:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}$$

Usage:

- $\mathcal{D}_{\text{train}}$: used to optimize model parameters θ
- $\mathcal{D}_{\text{val}}$: used to tune hyperparameters and detect overfitting
- $\mathcal{D}_{\text{test}}$: used for final unbiased evaluation



7. The i.i.d. Assumption

Why can we relate training error and test error? Because of the **i.i.d. assumption**:

- Each example (x_i, y_i) is drawn **independently** from the same distribution.
- Training and test sets are thus **identically distributed**.

$$(x_i, y_i) \sim p_{\text{data}}(x, y) \quad \text{for all } i$$

Implication:

- The empirical risk on the training set is a stochastic estimate of the true risk.
- This allows us to study mathematically how minimizing training loss impacts test loss.

9. Detecting Overfitting

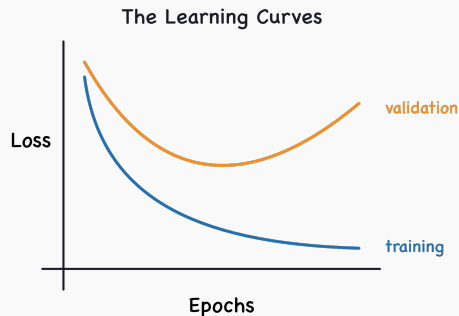
Loss functions during training:

$$\mathcal{L}_{\text{train}}(\theta) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x,y) \in \mathcal{D}_{\text{train}}} \ell(h_{\theta}(x), y)$$

$$\mathcal{L}_{\text{val}}(\theta) = \frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{(x,y) \in \mathcal{D}_{\text{val}}} \ell(h_{\theta}(x), y)$$

Observation:

- If $\mathcal{L}_{\text{train}} \downarrow$ and $\mathcal{L}_{\text{val}} \uparrow \rightarrow$ overfitting begins.
- Apply **early stopping**.



Training vs validation loss across epochs.

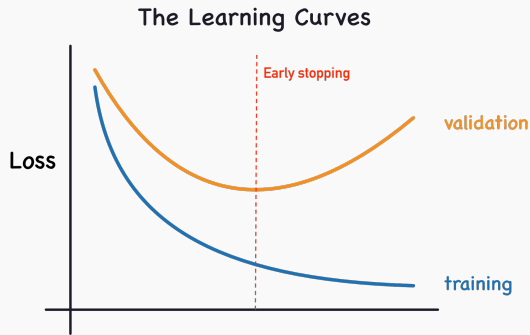
10. Early Stopping

Goal: prevent overfitting by stopping training when validation performance no longer improves.

Formal rule:

$$\mathcal{L}_{\text{val}}^{(t)} > \min_{k \leq t-p} \mathcal{L}_{\text{val}}^{(k)} \quad \text{for } p \text{ consecutive epochs}$$

Stop training after p epochs (patience) without improvement in validation loss.



Pseudo-code:

- **Initialize** `best_val_loss =  $+\infty$`
- **For each epoch** $t = 1, 2, \dots$
 - **Train model on** $\mathcal{D}_{\text{train}}$
 - **Compute** $\mathcal{L}_{\text{val}}^{(t)}$
 - **If** $\mathcal{L}_{\text{val}}^{(t)} < \text{best_val_loss}$:
 - **Save model weights**
 - `best_val_loss =  $\mathcal{L}_{\text{val}}^{(t)}$`
 - `no_improve = 0`
 - **Else:** `no_improve + 1`
 - **If** `no_improve >  $p$` : **stop training**

Regularization

13. Regularization — Definition and Motivation

“Regularization is any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error.”

(Goodfellow et al., *Deep Learning*, 2016)

12. Regularization: Controlling Model Complexity

Goal: prevent overfitting by adding a penalization term which limit the capacity of the model.

We modify the training objective:

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}_{\text{train}}(\theta) + \lambda \mathcal{R}(\theta)$$

where:

- $\mathcal{L}_{\text{train}}$: training loss,
- $\mathcal{R}(\theta)$: regularization penalty,
- λ : regularization hyperparameter controlling the strength of regularization.

Trade-off:

- $\lambda = 0$: risk of overfitting,
- λ too high: underfitting.

Regularization controls the model's complexity.

15. Should We Regularize Biases?

Question: *Why do we usually regularize the weights of a neural network but not the biases?*

15. Should We Regularize Biases?

Question: *Why do we usually regularize the weights of a neural network but not the biases?*

Answer:

- Each **weight** defines how two variables interact. Fitting it well requires observing both variables in many conditions.
- Each **bias** affects only one variable — it just shifts activations. It can usually be learned reliably from less data.
- Regularizing biases would therefore:
 - Add little benefit to generalization.
 - Risk introducing **underfitting**.

Reference: Goodfellow, Bengio, Courville — *Deep Learning*, MIT Press (2016), §7.1.1

13. Should Each Layer Have Its Own Regularization Coefficient?

Question: *In a neural network, should we use a separate regularization strength λ_ℓ for each layer?*

13. Should Each Layer Have Its Own Regularization Coefficient?

Question: *In a neural network, should we use a separate regularization strength λ_ℓ for each layer?*

Answer:

- **In theory:** Yes — each layer may have weights of different scales, sensitivities, or roles. Using distinct λ_ℓ allows finer control over how much each layer is regularized.
- **In practice:** Searching for many hyperparameters is costly. Using a single global λ reduces the hyperparameter search space while still offering good generalization.

Takeaway:

Using one regularization strength for all layers is a reasonable trade-off between optimality and practicality.

13. L2 Regularization (Weight Decay)

Definition:

$$\mathcal{L}_{L2}(\theta) = \mathcal{L}_{\text{train}}(\theta) + \frac{\lambda}{2} \|\theta\|_2^2 \quad \text{where} \quad \|\theta\|_2^2 = \sum_j \theta_j^2$$

Gradient:

$$\nabla_{\theta} \left(\frac{\lambda}{2} \|\theta\|_2^2 \right) = \lambda \theta$$

Update rule:

$$\theta \leftarrow (1 - \eta\lambda)\theta - \eta \nabla_{\theta} \mathcal{L}_{\text{train}}(\theta)$$

⇒ every weight is **decayed linearly** toward zero.

Effect:

- discourages large weights,
- encourages using all neurons “a little,”
- smooths decision boundaries.

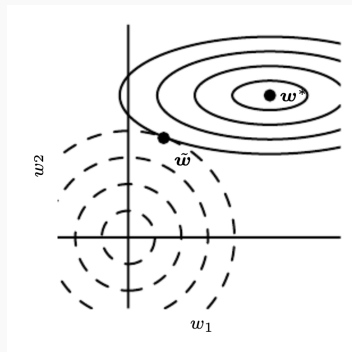


Figure: Solid ellipses = contours of unregularized loss $J(w)$;
dotted circles = L_2 penalty contours.
 w^* = unregularized minimum,
 $\tilde{w}$ = new optimum after regularization.

14. L1 Regularization (Sparsity)

Definition:

$$\mathcal{L}_{\text{L1}}(\theta) = \mathcal{L}_{\text{train}}(\theta) + \lambda \|\theta\|_1$$

where:

$$\|\theta\|_1 = \sum_j |\theta_j|$$

Gradient:

$$\nabla_{\theta}(\lambda \|\theta\|_1) = \lambda \text{sign}(\theta)$$

where $\text{sign}(\theta_j) = 1$ if $\theta_j > 0$, -1 if $\theta_j < 0$, and 0 if $\theta_j = 0$.

Update rule:

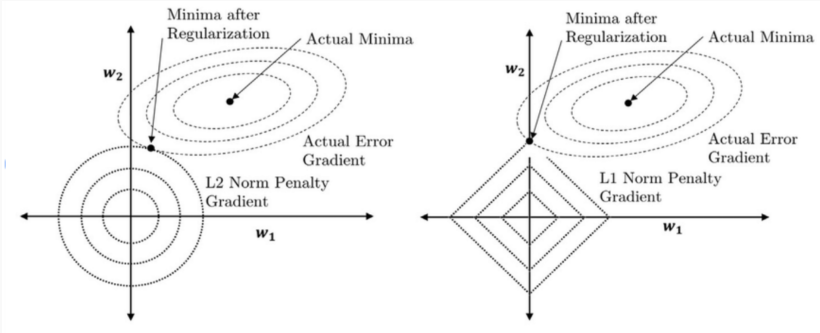
$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{\text{train}} - \eta \lambda \text{sign}(\theta)$$

Effect:

- drives many weights to exactly zero,
- creates **sparse models**,
- performs implicit feature selection.

16. L1 vs L2 Regularization — Comparison

Property	L1 (Lasso)	L2 (Ridge)
Penalty term	$\lambda \ \theta\ _1 = \lambda \sum_j \theta_j $	$\frac{\lambda}{2} \ \theta\ _2^2 = \frac{\lambda}{2} \sum_j \theta_j^2$
Gradient	$\lambda \text{sign}(\theta)$	$\lambda \theta$
Effect on weights	Many $\theta_j = 0$ (sparse)	All θ_j small (smooth)
Interpretation	Feature selection	Weight shrinkage
Differentiability	Non-differentiable at 0	Smooth everywhere
Typical use	Interpretability, sparsity	Stability, generalization



15. Max-Norm Constraint

Principle: Rather than adding a penalty to the loss, Max-Norm directly **constrains** the norm of each weight vector:

$$\|w_j^{(\ell)}\|_2 \leq c \quad \forall j, \ell$$

Reprojection step: After each update, weights violating the constraint are projected back:

$$w_j^{(\ell)} \leftarrow \frac{c}{\|w_j^{(\ell)}\|_2} w_j^{(\ell)}$$

Why it helps:

- Avoids *dead units* that arise with strong penalties (L1/L2).
- Prevents *exploding weights* and ensures stability for high learning rates.
- Encourages faster but bounded exploration of parameter space.

Unlike penalties, Max-Norm has no effect near the origin — it acts only when weights grow too large.

16. Dropout Regularization

Goal: prevent overfitting by reducing co-adaptation between neurons during training.

Principle: Randomly deactivate a fraction of neurons during training.

$$a_{\text{drop}}^{(\ell)} = m^{(\ell)} \odot a^{(\ell)}, \quad m_j^{(\ell)} \sim \text{Bernoulli}(p)$$

Each neuron is **kept** with probability p and **dropped** with probability $1 - p$.

Inverted dropout:

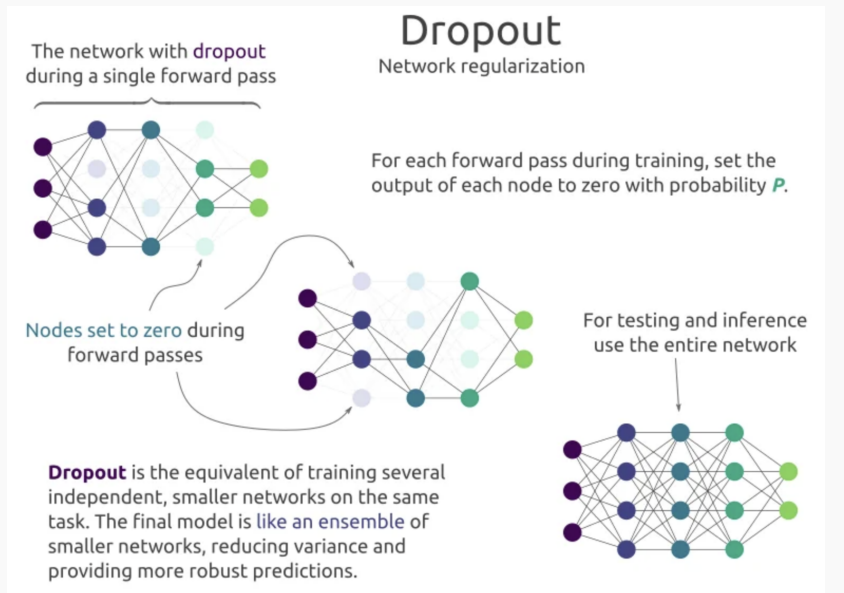
$$a_{\text{drop}}^{(\ell)} = \frac{m^{(\ell)}}{p} \odot a^{(\ell)}$$

so that $\mathbb{E}[a_{\text{drop}}^{(\ell)}] = a^{(\ell)}$, ensuring consistent output scales between training and test phases.

Effects:

- prevents reliance on specific neurons (decorrelation),
- acts as implicit model averaging (many subnetworks),
- improves generalization on unseen data.

17. Dropout Regularization — Illustration



https://www.reddit.com/r/learnmachinelearning/comments/x89qsi/dropout_in_neural_networks_while_training/lightbox

14. Choosing the Right Regularization

The No Free Lunch principle:

“There is no best machine learning algorithm, and, in particular, no best form of regularization.”

Implications:

- The effectiveness of a regularizer depends on the data, the task, and the model.
- Some regularizations are very general-purpose (e.g., weight decay, dropout, early stopping).
- Others encode domain-specific assumptions (e.g., sparsity, smoothness, invariances).

In practice:

“The best model can be a large one that has been regularized appropriately.”

5. Batch, Mini-Batch, and Pure SGD

Stochastic approximation

We estimate the gradient from a small random subset $\mathcal{B}$ of the data:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \ell(h_{\theta}(x_i), y_i)$$

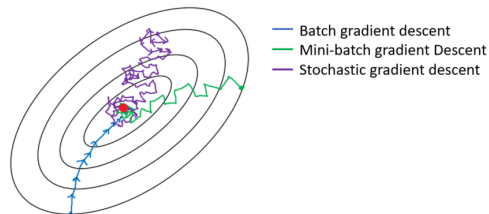
→ This is the **Stochastic Gradient Descent (SGD)** update rule.

The choice of $\mathcal{B}$ defines different methods:

- **Batch:** $\mathcal{B} = \{1, \dots, N\}$
- **Mini-Batch:** $1 < |\mathcal{B}| < N$
- **Pure SGD:** $|\mathcal{B}| = 1$

Comparison summary

Method	Speed	Noise
Batch	Slow	Low
Mini-Batch	Medium	Moderate
Pure SGD	Fast	High



Gradient noise decreases as batch size increases.

5. Batch, Mini-Batch, and Pure SGD

Stochastic approximation

We estimate the gradient from a small random subset $\mathcal{B}$ of the data:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \ell(h_{\theta}(x_i), y_i)$$

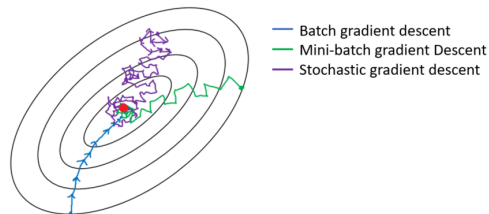
→ This is the **Stochastic Gradient Descent (SGD)** update rule.

The choice of $\mathcal{B}$ defines different methods:

- **Batch:** $\mathcal{B} = \{1, \dots, N\}$
- **Mini-Batch:** $1 < |\mathcal{B}| < N$
- **Pure SGD:** $|\mathcal{B}| = 1$

Comparison summary

Method	Speed	Noise
Batch	Slow	Low
Mini-Batch	Medium	Moderate
Pure SGD	Fast	High



Gradient noise decreases as batch size increases.

Data Augmentation

18. Data Augmentation — Principle

Goal: improve generalization by artificially enlarging the training set.

Instead of changing the model or the loss, **data augmentation** modifies the input data while preserving their semantic meaning.

$$(x, y) \longrightarrow (T(x), y)$$

where $T(\cdot)$ is a transformation applied to the data.

Examples of transformations:

- **Images:** rotations, flips, crops, color jitter, Gaussian noise.
- **Audio:** pitch shift, time stretch, additive noise.
- **Text:** synonym replacement, paraphrasing, word dropout.

Effect:

- increases data diversity without new labels,
- reduces overfitting,

19. Data Augmentation — Formalization

Let the training set be $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^N$.

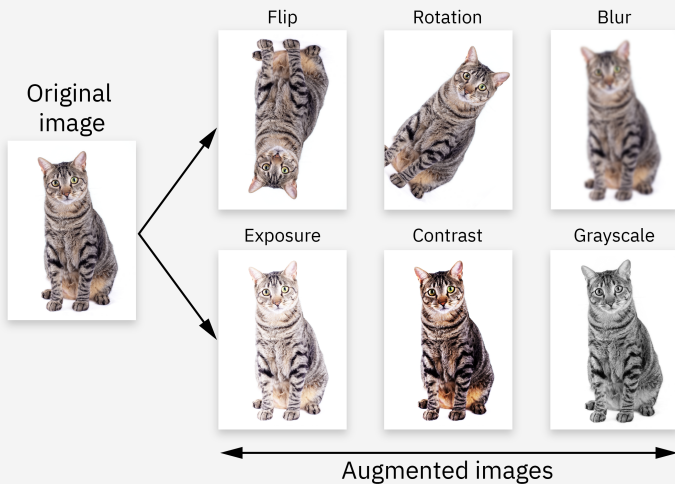
We introduce a family of transformations $T \sim \mathcal{T}$ and define the augmented loss:

$$\mathcal{L}_{\text{aug}}(\theta) = \frac{1}{N} \sum_{i=1}^N \frac{1}{N_t} \sum_{t=1}^{N_t} \ell(h_{\theta}(T_t(x_i)), y_i)$$

Interpretation:

- The model is trained on transformed versions of the inputs.
- In expectation, it learns features invariant to the chosen transformations.

20. Data Augmentation — Example



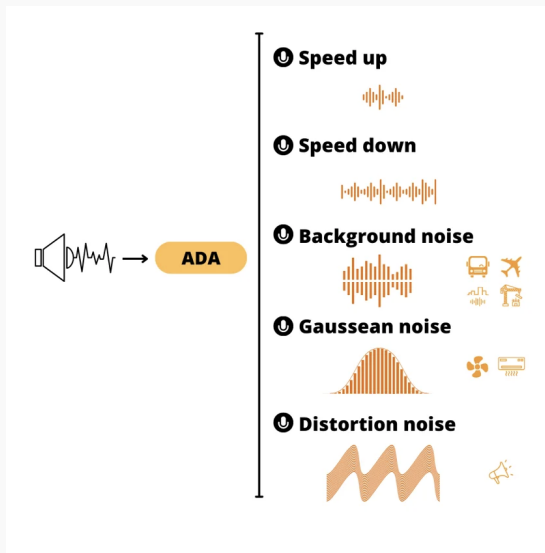
<https://www.ibm.com/fr-fr/think/topics/data-augmentation>

20. Data Augmentation — Example



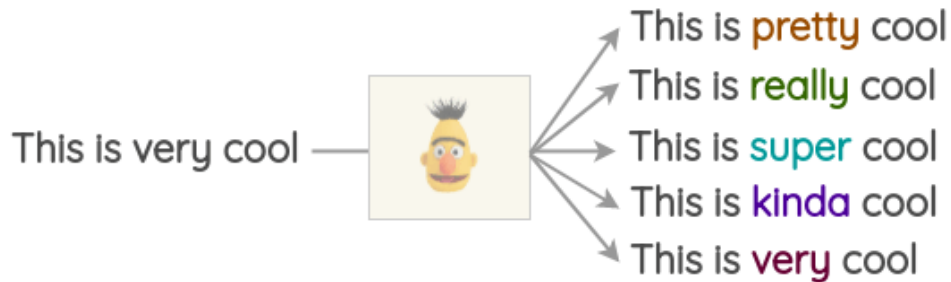
<https://www.ibm.com/fr-fr/think/topics/data-augmentation>

20. Data Augmentation — Example



<https://blog.pangeanic.com/audio-data-augmentation-techniques-and-methods>

20. Data Augmentation — Example



<https://amitnss.com/posts/data-augmentation-for-nlp>

10. Hyperparameter Tuning and Validation

Goal: find hyperparameters λ minimizing the validation loss:

$$\lambda^* = \arg \min_{\lambda} \mathcal{L}_{\text{val}}(\theta^*(\lambda))$$

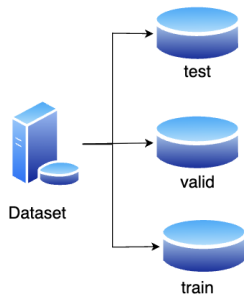
Common hyperparameters:

- Learning rate η
- Mini-batch size $|\mathcal{B}|$
- Regularization terms (weight decay, dropout, etc.)

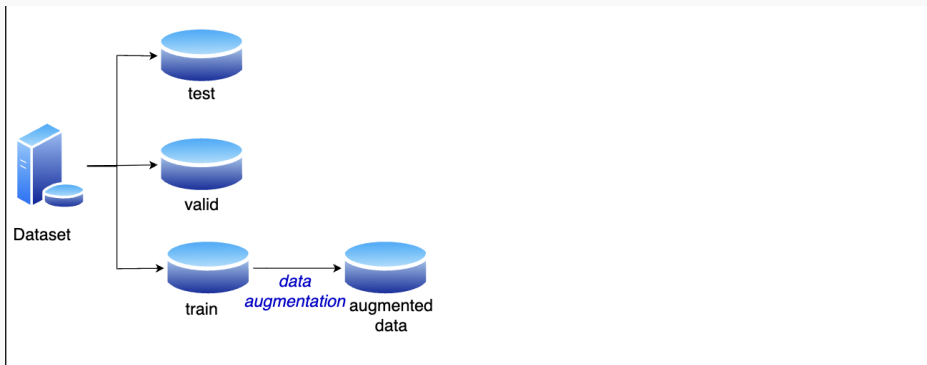
Methods:

- Grid Search — test all combinations.
- Random Search — random sampling of configurations.
- Bayesian Optimization — guided search.

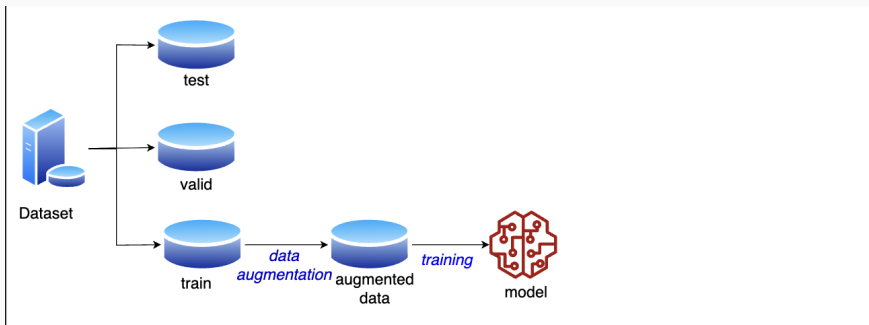
Full Workflow



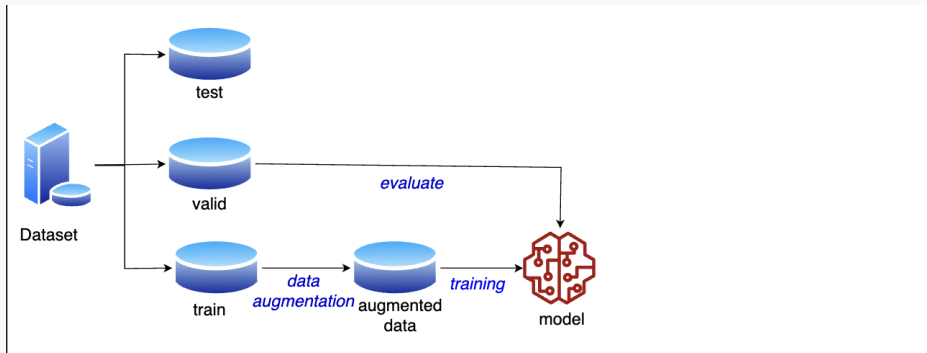
Full Workflow



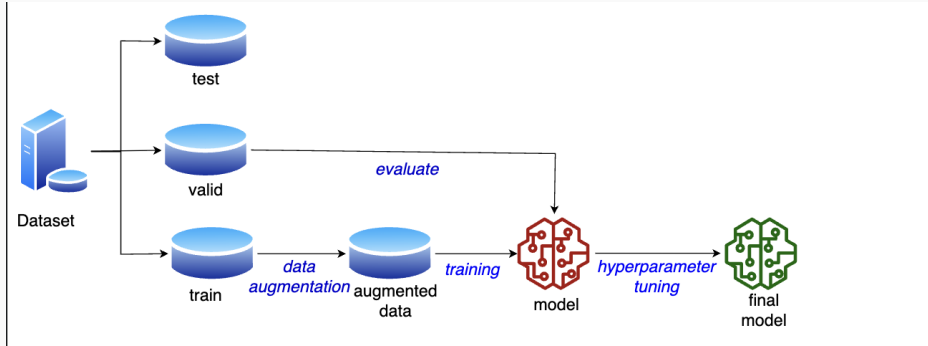
Full Workflow



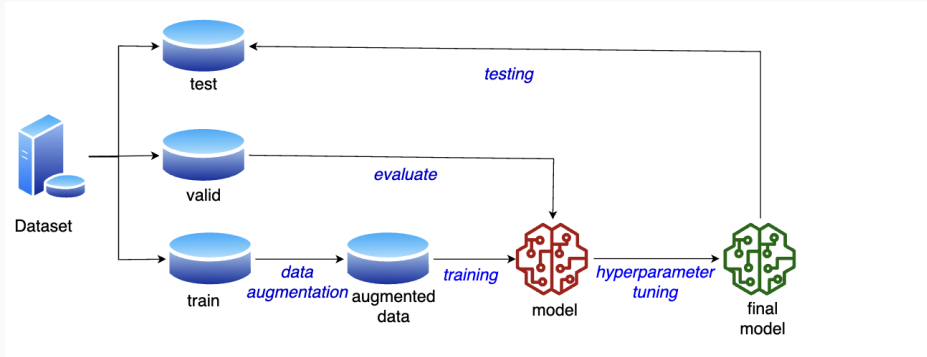
Full Workflow



Full Workflow



Full Workflow



The Challenges with Standard Optimizers

Challenges in Gradient-Based Optimization

We have seen the Gradient Descent update rule:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$$

However, navigating the loss landscape presents several challenges:

- Choosing the right learning rate η is difficult:

Challenges in Gradient-Based Optimization

We have seen the Gradient Descent update rule:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$$

However, navigating the loss landscape presents several challenges:

- Choosing the right learning rate η is difficult:
 - Too small: training is extremely slow.

The Challenges with Standard Optimizers

Challenges in Gradient-Based Optimization

We have seen the Gradient Descent update rule:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$$

However, navigating the loss landscape presents several challenges:

- Choosing the right learning rate η is difficult:
 - Too small: training is extremely slow.
 - Too large: training can overshoot the minimum and diverge.

Challenges in Gradient-Based Optimization

We have seen the Gradient Descent update rule:

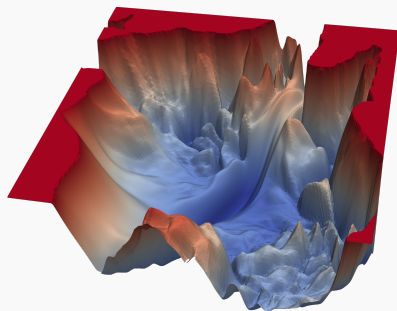
$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$$

However, navigating the loss landscape presents several challenges:

- Choosing the right learning rate η is difficult:
 - Too small: training is extremely slow.
 - Too large: training can overshoot the minimum and diverge.
- A single learning rate for all parameters is suboptimal.
Some parameters may need larger updates while others require smaller, finer adjustments.

Challenges in Gradient-Based Optimization

- The loss landscape can contain *ravines* and *saddle points*, where the gradient is very small in one direction and very large in another, causing standard GD to oscillate or get stuck.



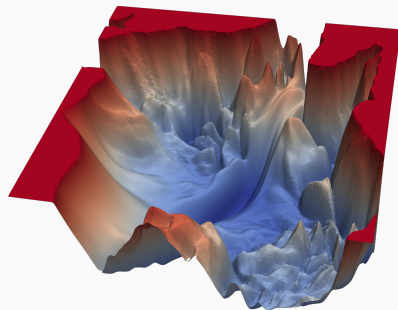
Loss landscape: loss surface $\mathcal{L}(\theta)$ along two directions in parameter space around a local minimum.

The Challenges with Standard Optimizers

Challenges in Gradient-Based Optimization

- The loss landscape can contain *ravines* and *saddle points*, where the gradient is very small in one direction and very large in another, causing standard GD to oscillate or get stuck.

Goal: Find algorithms that adapt to the geometry of the loss function and accelerate convergence.



Loss landscape: loss surface $\mathcal{L}(\theta)$ along two directions in parameter space around a local minimum.

Batch vs. Stochastic Gradient Descent : The Need for Speed

The standard Batch Gradient Descent algorithm computes the gradient using the entire training dataset for a single parameter update:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta) = \theta - \eta \nabla_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(h_{\theta}(x_i), y_i)$$

Problem: this approach is computationally expensive and slow on large datasets,.

Stochastic Gradient Descent (SGD) offers a solution by estimating the gradient using a small, random subset of the data called a mini-batch B :

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \frac{1}{|B|} \sum_{i \in B} \ell(h_{\theta}(x_i), y_i)$$

- The noise in SGD updates can help escape shallow local minima, but it also creates an unstable convergence path.

Accelerating Gradient Descent with Momentum:

Problem: standard SGD oscillates in narrow ravines, slowing down convergence.

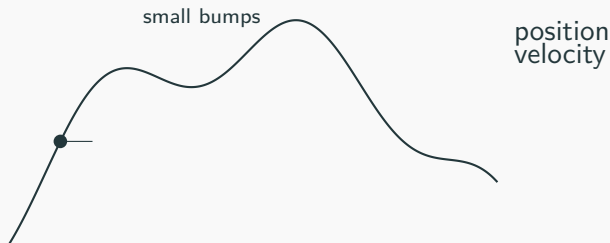
Idea: introduce momentum to accelerate movement in the relevant direction and dampen oscillations.

Analogy: a heavy ball rolling down a hill. It accumulates velocity, helping it to smooth out its path and power through small bumps or flat regions.

Momentum as a Heavy Ball Rolling Down a Hill

Analogy: a heavy ball rolls down a hill and accumulates velocity.

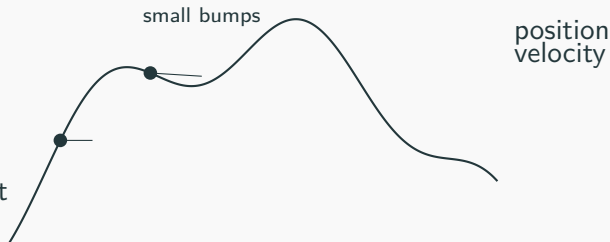
- Initially, movement is small on flat regions or small bumps.



Momentum as a Heavy Ball Rolling Down a Hill

Analogy: a heavy ball rolls down a hill and accumulates velocity.

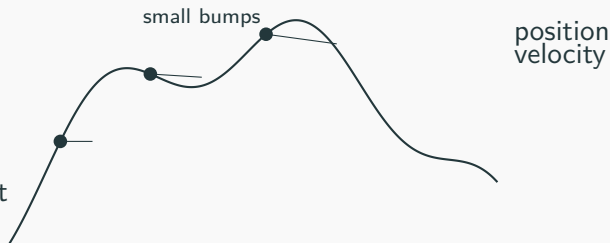
- Initially, movement is small on flat regions or small bumps.
- Velocity builds up in a consistent direction (momentum).



Momentum as a Heavy Ball Rolling Down a Hill

Analogy: a heavy ball rolls down a hill and accumulates velocity.

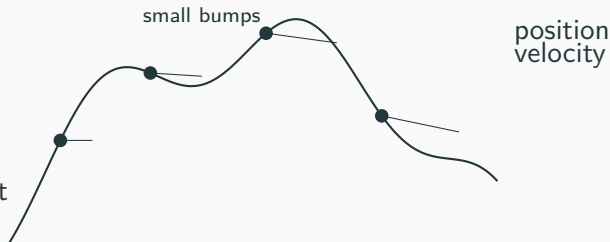
- Initially, movement is small on flat regions or small bumps.
- Velocity builds up in a consistent direction (momentum).
- The ball powers through small bumps instead of oscillating.



Momentum as a Heavy Ball Rolling Down a Hill

Analogy: a heavy ball rolls down a hill and accumulates velocity.

- Initially, movement is small on flat regions or small bumps.
- Velocity builds up in a consistent direction (momentum).
- The ball powers through small bumps instead of oscillating.



SGD with Momentum

The update involves a velocity vector v_t that accumulates an exponentially decaying moving average of past gradients.

Update rules:

$$v_t \leftarrow \beta v_{t-1} + \eta \nabla_{\theta} \mathcal{L}(\theta_{t-1})$$

$$\theta_t \leftarrow \theta_{t-1} - v_t$$

where β is the momentum coefficient (typically around 0.9).

Effect: the algorithm moves faster and more consistently towards the minimum, effectively smoothing the update trajectory.

Adaptive Gradient Algorithm (Adagrad)

Problem: some parameters (for example, those associated with rare features) are updated infrequently and should have a larger learning rate, while frequently updated parameters need smaller steps.

Idea: adapt the learning rate for each parameter individually based on its entire update history.

- Give larger updates to infrequent parameters.
- Give smaller updates to frequent parameters.

Mechanism: Adagrad accumulates the sum of squared gradients for each parameter and uses this to scale the learning rate.

Adagrad: Per-Parameter Learning Rates

Update rule (for parameter θ_i):

$$\theta_{t+1,i} \leftarrow \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii}} + \varepsilon} \nabla_{\theta_i} \mathcal{L}(\theta_t)$$

where G_t is a diagonal matrix containing the sum of the squares of past gradients for each parameter θ_i .

Advantage: excellent for sparse data (e.g., in natural language processing).

Drawback: the accumulated sum of squares in the denominator only grows, causing the learning rate to become very small and potentially stopping training prematurely.

RMSProp: Fixing AdaGrad's Decaying Learning Rate

Problem: AdaGrad's learning rate decays too aggressively, which can halt learning.

Idea: instead of accumulating all past squared gradients, RMSProp uses an exponentially decaying moving average. This prevents the denominator from growing indefinitely.

Mechanism: it gives more weight to recent gradients and "forgets" older ones, keeping the learning rate adaptive and responsive.

RMSProp: Fixing AdaGrad's Decaying Learning Rate

Let S_t be the moving average of squared gradients.

Update rules:

$$\begin{aligned} S_t &\leftarrow \beta S_{t-1} + (1 - \beta) (\nabla_{\theta} \mathcal{L}(\theta_t))^2 \\ \theta_{t+1} &\leftarrow \theta_t - \frac{\eta}{\sqrt{S_t} + \varepsilon} \nabla_{\theta} \mathcal{L}(\theta_t) \end{aligned}$$

where β is the decay rate hyperparameter (typically around 0.99).

Effect: RMSProp maintains the adaptivity of AdaGrad while avoiding vanishing learning rates, allowing continued and stable convergence.

Adam: The Best of Both worlds

Adam combines momentum (first moment) and RMSProp-like adaptivity (second moment).

It maintains two moving averages for each parameter:

- first moment (velocity): exponentially decaying average of past gradients m_t
- second moment (scale): exponentially decaying average of past squared gradients v_t

Adam: Adaptive Moment Estimation

Let $g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$. The updates are:

$$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t \qquad v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Parameter update:

$$\theta_{t+1} \leftarrow \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

Typical choices: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$. Adam is efficient and robust across many deep learning models.

Summary and Practical Recommendations

There is no single best optimizer for all problems (no free lunch). Choose based on data regime, model size, and tuning budget.

Optimizer	Key idea	Pros	Cons
SGD	Simple gradient update	Good generalization, low memory	Slow; sensitive to learning rate; oscillations in ravines
Momentum	Adds velocity to SGD	Faster convergence; dampens noise	Can overshoot; one extra hyper-parameter (β)
RMSProp	Adaptive learning rates with exponential decay	Fixes AdaGrad's decay issue; fast convergence	Still sensitive to global learning rate η
Adam	Combines momentum and RMSProp	Fast, robust, effective; good default	Sometimes slightly worse generalization than well-tuned SGD

Practical tips:

- start with Adam; if final generalization matters most and you can tune, try switching to SGD (with momentum) late in training
- always tune η ; use learning-rate schedules or warmup

For Going Further — Foundational References

Foundational papers (one per notion)

- **Backpropagation** — D. E. Rumelhart, G. E. Hinton, R. J. Williams, *Nature*, 1986.
- **SGD (stochastic approximation)** — H. Robbins, S. Monro, *Proc. Cambridge Phil. Soc.*, 1951.
- **Acceleration (Nesterov)** — Y. Nesterov, *Soviet Math. Dokl.*, 1983.
- **Adam (adaptive)** — D. P. Kingma, J. Ba, *ICLR*, 2015.
- **Batch Normalization** — S. Ioffe, C. Szegedy, *ICML*, 2015.
- **Dropout** — N. Srivastava et al., *JMLR*, 2014.
- **Early Stopping** — L. Prechelt, *Neural Networks: Tricks of the Trade*, 1998.
- **Weight Decay / L2** — A. Krogh, J. A. Hertz, *NIPS*, 1992.
- **Data Augmentation (vision)** — P. Y. Simard et al., *ICDAR*, 2003.
- **Hyperparam — Random Search** — J. Bergstra, Y. Bengio, *JMLR*, 2012.
- **Hyperparam — Bayesian Opt.** — J. Snoek, H. Larochelle, R. Adams, *NeurIPS*, 2012.

Deep Learning — textbooks

- I. Goodfellow, Y. Bengio, A. Courville — *Deep Learning*, MIT Press, 2016.
- (C. M. Bishop — *Pattern Recognition and Machine Learning*, Springer, 2006.)
- K. P. Murphy — *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022.

Optimization — books & notes

- S. Boyd, L. Vandenberghe — *Convex Optimization*, Cambridge Univ. Press, 2004.
- S. Bubeck — *Convex Optimization: Algorithms and Complexity*, Foundations and Trends in ML, 2015.
- Y. Nesterov — *Introductory Lectures on Convex Optimization*, Springer, 2004.
- (Course notes) *Convex Optimization: Fall 2019* : <https://www.stat.cmu.edu/~ryantibs/convexopt/>

appendix

5. Why the Loss is a Sum over the Samples

Loss for one training example:

$$\ell(h_{\theta}(x_i), y_i)$$

measures how far the model prediction $h_{\theta}(x_i)$ is from the true label y_i .

To train the model, we need to minimize the total error on the entire dataset:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(h_{\theta}(x_i), y_i)$$

Why the sum (or average)?

- Each sample (x_i, y_i) contributes to the total training error.
- Using the mean avoids dependence on the dataset size N .

Examples of individual loss functions:

$$\text{MSE: } \ell(h_{\theta}(x_i), y_i) = \frac{1}{2}(h_{\theta}(x_i) - y_i)^2 \quad \text{and} \quad \text{Cross-entropy: } \ell = -y_i \log h_{\theta}(x_i) - (1 - y_i) \log(1 - h_{\theta}(x_i))$$

Gradient used in training:

$$\nabla_{\theta} \mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \ell(h_{\theta}(x_i), y_i)$$

1. Theorem — Linearity Without Activations

Statement. Let an L -layer neural network be defined by

$$a^{(0)} = x, \quad a^{(\ell)} = \sigma(W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}), \quad h_{\theta}(x) = a^{(L)}.$$

If all activations are the identity function $\sigma(z) = z$, then

$$h_{\theta}(x) = \tilde{W}x + \tilde{b},$$

for some $\tilde{W}, \tilde{b}$ depending on $\{W^{(\ell)}, b^{(\ell)}\}$.

To prove:

- The composition of affine (linear + bias) transformations is itself affine.
- Hence, a deep network without nonlinearities collapses to one linear model.
- Nonlinear activations are required to approximate non-linear functions.

2. Theorem — Convergence of Gradient Descent

Assumptions. $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$ is convex, differentiable, and L -smooth:

$$\|\nabla \mathcal{L}(x) - \nabla \mathcal{L}(y)\| \leq L\|x - y\|.$$

Gradient Descent iteration:

$$\theta_{k+1} = \theta_k - \eta \nabla \mathcal{L}(\theta_k), \quad \text{with step size } \eta \leq \frac{1}{L}.$$

To prove:

$$\mathcal{L}(\theta_k) - \mathcal{L}(\theta^*) \leq \frac{\|\theta_0 - \theta^*\|^2}{2\eta k}.$$

Reference (proof). The complete proof can be found in:

- S. Bubeck, *Convex Optimization: Algorithms and Complexity*, arXiv:1405.4980, Section 3.2 “Gradient descent for smooth functions”, pages **26–29** (corresponding to pp. 266–269 in the compiled version).

3. Theorem — Convexity of L2-Regularized Objective

Assumptions. $L(\theta)$ is convex, and $\lambda \geq 0$.

Objective:

$$J(\theta) = L(\theta) + \frac{\lambda}{2} \|\theta\|_2^2.$$

To prove:

- $J(\theta)$ is convex for all $\lambda \geq 0$.
- If $\lambda > 0$, then $J(\theta)$ is λ -strongly convex.

Where to find the proof & statements.

- Boyd & Vandenberghe (2004), *Convex Optimization*:
 - First/second-order characterizations of convexity (Chap. 3, p. 83–84).
 - *Strong convexity* via Hessian bound $\nabla^2 f(x) \succeq mI$ and inequality $f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{m}{2} \|y - x\|^2$ (Sec. 9.1.2, p. 472).
- Bubeck (2015), *Convex Optimization: Algorithms and Complexity*:
 - Définition/équivalence : f est α -fortement convexe $\iff x \mapsto f(x) - \frac{\alpha}{2} \|x\|^2$ est convexe (Sec. 3.4, p. 48).
 - Application directe à $J(\theta) = L(\theta) + \frac{\lambda}{2} \|\theta\|^2$: si L est convexe, alors J est λ -fortement convexe.

4. Theorem — Bias–Variance Decomposition

Setup. For a predictor $\hat{f}(x; \mathcal{D})$ trained on dataset $\mathcal{D}$, and target $y = f^*(x) + \varepsilon$ with $\mathbb{E}[\varepsilon|x] = 0$, $\text{Var}(\varepsilon|x) = \sigma^2$.

Goal:

$$\mathbb{E}_{\mathcal{D}, \varepsilon}[(\hat{f}(x) - y)^2] = \underbrace{(\mathbb{E}_{\mathcal{D}}[\hat{f}(x)] - f^*(x))^2}_{\text{Bias}^2} + \underbrace{\text{Var}_{\mathcal{D}}(\hat{f}(x))}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Noise}}.$$

To prove:

- Expand $(\hat{f} - y)^2$ by adding and subtracting $\mathbb{E}_{\mathcal{D}}[\hat{f}(x)]$.
- Show that cross-terms vanish using independence and zero-mean noise.
- Interpret bias, variance, and irreducible noise analytically.

Interpretation: Overfitting $\Rightarrow$ low bias, high variance.

References (proof).

- T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer — §2.9 *Bias–Variance Tradeoff*.
- K. P. Murphy, *Machine Learning: A Probabilistic Perspective*, MIT Press — §7.5 *Bias–variance decomposition*.

5. Theorem — Effect of L2 Regularization (Weight Decay)

Objective:

$$J(\theta) = L_{\text{train}}(\theta) + \frac{\lambda}{2} \|\theta\|_2^2.$$

To prove:

- $\nabla_{\theta} \frac{\lambda}{2} \|\theta\|_2^2 = \lambda \theta$.
- The gradient descent update becomes:

$$\theta \leftarrow (1 - \eta\lambda) \theta - \eta \nabla_{\theta} L_{\text{train}}(\theta).$$

- This corresponds to **weight decay**: each update shrinks parameters by $(1 - \eta\lambda)$.

Implication: Regularization acts both as a norm penalty (theoretical view) and as multiplicative weight shrinkage (practical optimizer view).

3. Proof: Without Activations, the Network is Linear

Let an L -layer network be defined recursively:

$$a^{(0)} = x, \quad a^{(\ell)} = \sigma\left(W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}\right), \quad h_{\theta}(x) = a^{(L)}.$$

Assume all activations are identity: $\sigma(z) = z$.

Then each layer is linear:

$$a^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}.$$

Expanding recursively:

$$a^{(1)} = W^{(1)} x + b^{(1)},$$

$$a^{(2)} = W^{(2)} W^{(1)} x + W^{(2)} b^{(1)} + b^{(2)},$$

$$a^{(3)} = W^{(3)} W^{(2)} W^{(1)} x + W^{(3)} W^{(2)} b^{(1)} + W^{(3)} b^{(2)} + b^{(3)},$$

$\vdots$

$$a^{(L)} = \left(\prod_{\ell=L}^1 W^{(\ell)}\right) x + \sum_{k=1}^L \left(\prod_{\ell=L}^{k+1} W^{(\ell)}\right) b^{(k)}.$$

We define:

$$\tilde{W} = \prod_{\ell=L}^1 W^{(\ell)}, \quad \tilde{b} = \sum_{k=1}^L \left(\prod_{\ell=L}^{k+1} W^{(\ell)}\right) b^{(k)},$$

where an empty product is the identity matrix.

Hence:

$$h_{\theta}(x) = \tilde{W}x + \tilde{b}.$$

Vocabulary Summary — Key Concepts in Deep Learning

Term	Definition / Meaning
Neural Network Structure	
Neuron	Unit applying an activation to a weighted sum of inputs.
Layer	Group of neurons sharing same input/output dimensions.
Activation function	Non-linear function (ReLU, sigmoid, tanh, etc.).
Parameter (θ)	All trainable weights and biases in the network.
Forward pass	Compute output $h_{\theta}(x)$ for given input x .
Backward pass	Compute gradients via backpropagation.
Backpropagation	Chain-rule algorithm for efficient gradient computation.
Weight decay	Linear shrinkage of parameters via L2 regularization.
Learning rate (η)	Step size for parameter updates.
Optimizer	Update method (SGD, Adam, RMSProp, etc.).
Training, Generalization, and Regularization	
Loss function	Measures prediction error (MSE, cross-entropy).
Regularization	Prevents overfitting (L1, L2, dropout, data augm.).
Dropout	Randomly deactivates neurons during training.
Data augmentation	Expands dataset via label-preserving transforms.
Early stopping	Stops training when val. loss stops improving.
Over/Underfitting	Too complex / too simple model.
Bias-Variance trade-off	Balance between bias and variance.
Generalization	Ability to perform well on unseen data.
Validation/Test set	Used to tune or evaluate the model.
Epoch	One full pass over training data.
Batch/Mini-batch	Subset used for each gradient update.
Analytical and Statistical Notions	
Convex function	Function ensuring any local min. is global.
Strong convexity	Convexity with curvature $\lambda > 0$.
Lipschitz gradient	$\ \nabla f(x) - \nabla f(y)\ \leq L\ x - y\ $.
Gradient norm	$\ \nabla_{\theta} \mathcal{L}\ $ = loss steepness.
Bias	Expected error from true target.
Variance	Model sensitivity to dataset variations.
Irreducible noise (σ^2)	Data randomness independent of model.
Convex combination	$tx_1 + (1 - t)x_2$, with $t \in [0, 1]$.
Coercive function	$\rightarrow \infty$ as $\ \theta\ \rightarrow \infty$ (ensures minima).

Notation Summary — Analytical Symbols

Symbol	Meaning
$x_i \in \mathcal{X}$	Input sample (feature vector).
$y_i \in \mathcal{Y}$	Corresponding target label.
$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$	Training dataset of N examples.
$h_\theta(x)$	Neural network output for input x with parameters θ .
$\theta = \{W^{(\ell)}, b^{(\ell)}\}_{\ell=1}^L$	All weights and biases of an L -layer network.
$W^{(\ell)}, b^{(\ell)}$	Weight matrix and bias vector of layer ℓ .
$a^{(\ell)}, z^{(\ell)}$	Activation and pre-activation at layer ℓ .
$\sigma(\cdot)$	Activation function (e.g., ReLU, tanh, sigmoid).
$\ell(h_\theta(x_i), y_i)$	Individual loss for one sample.
$\mathcal{L}(\theta)$	Empirical loss (average over the dataset).
$\nabla_\theta \mathcal{L}(\theta)$	Gradient of loss w.r.t. parameters.
η	Learning rate in gradient descent.
λ	Regularization coefficient (strength).
$\ \theta\ _1, \ \theta\ _2$	L1 and L2 norms of parameters.
$\mathcal{L}_{\text{train}}, \mathcal{L}_{\text{val}}$	Training and validation losses.
$f^*(x)$	True function (data-generating process).
$\hat{f}(x; \mathcal{D})$	Model prediction learned from dataset $\mathcal{D}$.
ε	Random noise term with $\mathbb{E}[\varepsilon x] = 0$.
$\text{Bias}^2, \text{Var}, \sigma^2$	Components of MSE decomposition.